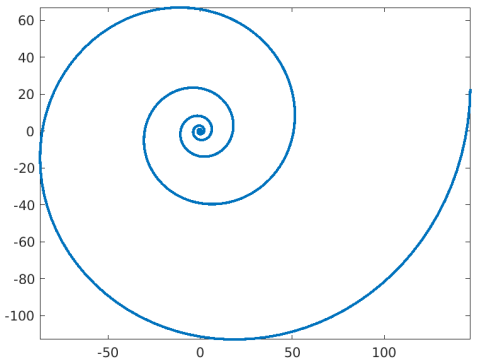
CS319: Scientific Computing (with MATLAB)

DRAFT: Figures and Fitting

Niall Madden

Week 5: **9am and 4pm**, 08 Feb 2023



Important: you should read:

- Chapter 5 of Learning MATLAB: https://doi-org.nuigalway.idm.oclc.org/10.1137/1.9780898717662
- Chapter 8 of The MATLAB Guide: https://doi-org.nuigalway.idm.oclc.org/10.1137/1.9781611974669

This week, in CS319:

1. 1: Matrix Functions
   - Recap
   - Other matrix functions
2. 2: Matrix division
3. 3: 2D Graphics
   - Log plots
4. 4: 3D Graphics
5. 4: Approximation
   - Taylor Approximation

Preview

At a glance, this week's class is all about plotting functions, in 1 and 2 dimensions.

But really it is about using such plots to get a better understanding of the problems we are working on.

For example, we will

- Use a plot to estimate the computational complexity of a linear solver;
- Investigate a least-squares problem

Last week we learned about working with matrices in MATLAB:

- You can define a matrix by listing its entries between square brackets. List entries by row, with a comma (or space) between columns, and a semicolon between rows.
- Use round brackets to access entries. Indexing is from 1. E.g., `A(2,4)` returns the entry in row 2, column 4 of A (assuming it exists). Possible errors:
  - `Unrecognized function or variable 'A'.`
  - `Index in position 1 exceeds array bounds. Index must not exceed 1.`
- Similarly, you can set an entry this way. E.g., `B(2,3)=4` sets the entry of B in row 2, column 2 to 4. If B does not exist, then it is created, with all entries, except `B(2,3)` set to zero.
- Vector indexing works as for vectors. E.g,
  - `A(1:2, 1:2)` refers to the 2×2 leading principle submatrix of A.
  - `A(:,3)` refers to all of column 3 of A.
- You can add, subtract, and multiply matrices using the usual arithmetic operators `+`, `-`, and `*`, respectively.
- "dot" operations are entry-wise.

- `inv(A)`
- `det(A)`
- `A'` is the transpose of A
- `eig(A)` estimate the eigenvalues and eigenvectors of A.
- `diag()` is a somewhat unusual function. Given an matrix as its argument, A, it returns the vector of diagonal entries of A.
  Given a vector as its argument, it returns a diagonal matrix with the vectors entries as its diagonal.
  Note: `B = diag(diag(A))` can be very useful. What do you think it does?

- `tril(A)`, `tril(A,k)`, `triu(A)`) and `tril(A,k)`.

And there are lots of other functions that you may have met in a linear algebra module, but we wait until we need them.

2: Matrix division

For scalars (i.e, 1×1 matrices), "division" is well understood: we know what `a/b` means $\frac{a}{b} = ab^{-1}$. This is called "right division" in MATLAB.

MATLAB also has "left division": `a\b` means $\frac{b}{a} = a^{-1}b$.

The reason for this, is that, if `A` is a matrix, and `b` and `x` are vectors so that $Ax = b$, then, of course $x = A^{-1}b$.

Solving $Ax = b$

In MATLAB, if you are given a matrix A and vector b, then we usually solve $Ax = b$ with "backslash":

```
>> x = A\b
```

2: Matrix division

It is important to note that the "backslash" operator is highly optimised. And it does not compute the inverse of a matrix. More likely, it uses Gaussian elimination, or some variant (depends on the matrix).

MATLAB can invert a matrix, using the `inv(A)` function. But if you just wish to solve a linear system, backslash is is faster, and uses much less memory.

2: Matrix division

SolverTimerV01.m

```
for n=2.^(2:11)
    A = randn(n); b = ones(n,1);

    % Test matrix left divide
    mld_start = tic;
    x = A\b;
    mld_time = toc(mld_start);

    % Test inv()
    inv_start = tic;
    B = inv(A);
    x = B*b;
    inv_time = toc(mld_start);

    fprintf('n=%4d. MLD time=%6.3fs, inv time=%6.4fs (Speed
        up = %5.2f)\n', ...
       n, mld_time, inv_time, inv_time/mld_time);
end
```

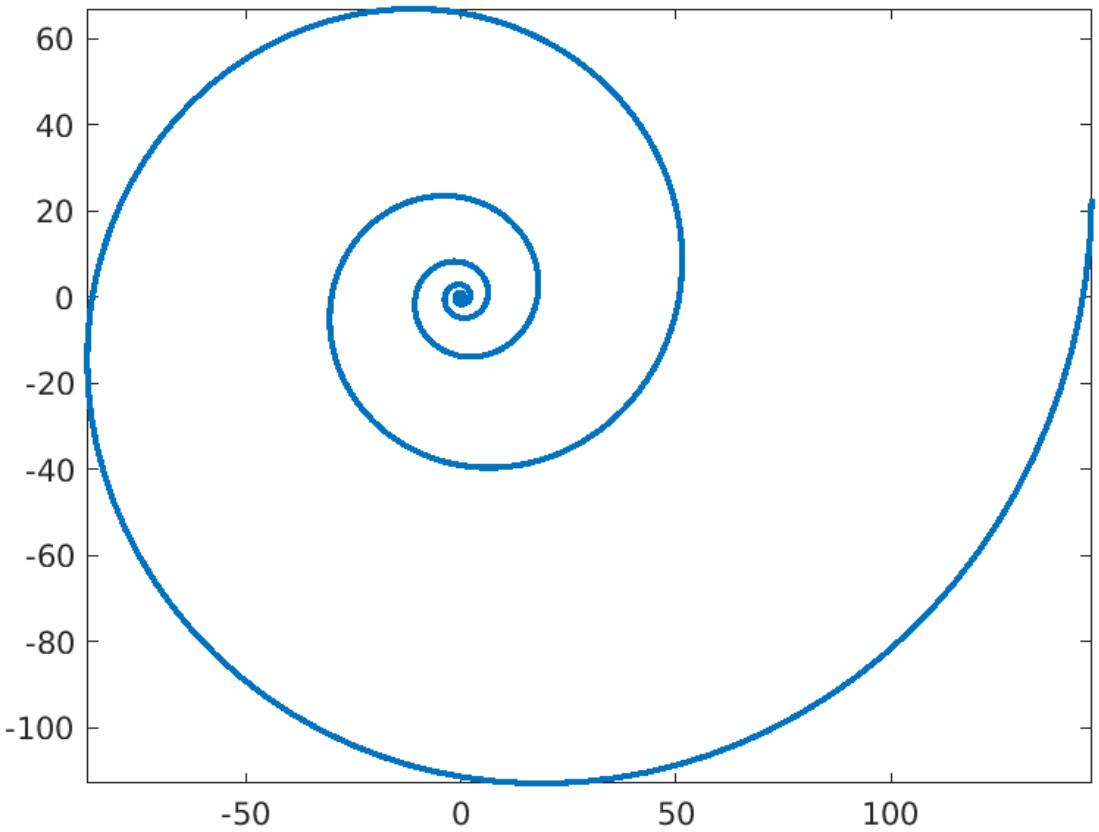
Try this. I get a speed-up of a factor of about 2.5. We'll return to this example later...

3: 2D Graphics

MATLAB is good at visualising functions and data. We have already seen some examples, such as `fplot` and `fsurf`, last week.

`fplot` is quite versatile, and can plot implicit functions:

```
>> fplot(@(t)exp(-t).*sin(6*t), @(t)exp(-t).*cos(6*t))
```



3: 2D Graphics

However, plotting such functions is relatively rare. It is more common to plot data points. For example, we'll plot US census population data, which comes with MATLAB for illustration purposes.

The `plot(x,y)` plots the vectors x and y, which must have the same number of entries, joining the points (x_1, y_1), (x_2, y_2), ..., (x_n, y_n).

Syntax:

Other examples Try these, and see the differences:

```
load census; % loads vectors cdate and pop
plot(cdate,pop)

plot(cdate,pop,'ro') % red circles

plot(cdate,pop,'k:') % black dotted lines

plot(cdate,pop,'ms--')

plot(cdate,pop,'ms--', 'LineWidth', 3, 'MarkerSize', 10)
```

3: 2D Graphics

Some useful functions for enhancing images:

```
title('string')

legend('string')

xlabel('string')

ylabel('string')

grid on
```

Also useful:

```
figure();

figure(n);

hold on;
hold off;

subplot(r,c,n)
```

One important application of plotting is to observe the growth of functions.

Example

In Section 2 we computed the time taken to solve linear systems by two different methods. Suppose we know that these times are polynomials in n, the order of the matrix. How can we determine to degree of the polynomials?

Here is how to proceed.

1. When we say T is a polynomial in n, we mean that

$$T(n) = c_0 + c_1 n + c_2 n^2 + \cdots + c_p n^p.$$

   Our goal is to find a likely value for p. (Less import, but we'll also estimate c_p).

2. Usually, n is very big. So we can approximate T as

$$T(n) \approx c_p n^p.$$

3. Taking the logarithm we get

$$\log\big(T(n)\big) = \log(c_p) + p\log(n).$$

4. So p is the slope of the line when plotting $\log(T(n))$ against $\log(n)$. This is such a common task, that there is a built-in function, `loglog()` to do this.

Once plotted, we can use trial-and-error to work out the slope. The easiest/laziest approach would be to successively include plots to cN, cN^2, cN^3, until one of them looks good. Choose c so that this line agrees with one of the data points (e.g., the last one).

SolverTimerV02.m

```
for n=2.^(4:13)
    k=k+1;
    Ns(k) = n; % Should really pre-allocate
    A = randn(n,n);
    b = ones(n,1);

    % Test matrix left divide
    mld_start = tic;
    x = A\b;x = A\b;
    mld_time(k) = toc(mld_start);

    % Test inv()
    inv_start = tic;
    B = inv(A); x = B*b;
    inv_time(k) = toc(mld_start);
end
c = mld_time(end)/Ns(end)^3;
loglog(Ns, inv_time, '--o', Ns, mld_time, '-.d', ...
    Ns, c*Ns.^3, '-k','LineWidth', 2, 'MarkerSize', 10)
```

One of the reasons that "backslash" is so much faster than `inv()` is that it first analyses the matrix to see if there are any short-cuts. Eventually it falls back on using Gaussian Elimination (or equivalent).

To demonstrate that, and to use some functions we mentioned earlier, let's apply the same approach to solving a linear system where the matrix is triangular.

(See notes from class for explanation and final code).

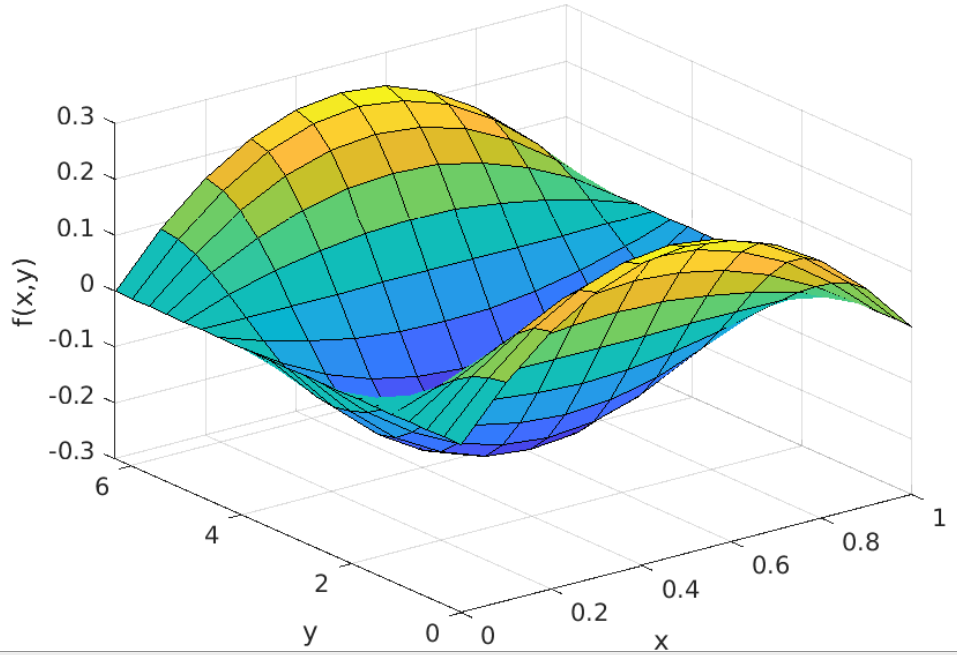
Log plots

4: 3D Graphics

Plotting surfaces is slightly more complicated than 2D plots. Often, one needs to create matrices, X and Y, Z where $Z(i,j) = f(X(i,j), Y(i,J))$.

Plot3D_demo.m

```
f = @(x,y)x.*(1-x).*cos(y);
x = linspace(0,1,11);
y = linspace(0,2*pi,21);
[X,Y]=meshgrid(x,y); % semi-colon is important here!
surf(X,Y,f(X,Y))
```



4: 3D Graphics

To explain this code...

Plot3D_demo.m

```
f = @(x,y)x.*(1-x).*cos(y);
x = linspace(0,1,11);
y = linspace(0,2*pi,21);
[X,Y]=meshgrid(x,y); % semi-colon is important here!
surf(X,Y,f(X,Y))
```

4: 3D Graphics

Some variation is possible. For example...

- Try using `grid()` instead of `surf()`.
- Once plotted, you can change the shading using one of
  `shading flat` `shading faceted` `shading interp`
- Very useful: use `colormap` to select the colours used. There are lots of options but the most useful are `hot`, `cool`, `gray`, `autumn`, `winter`, `summer` and `spring`.

One of the key concepts of Scientific Computing is approximation of "complicated" functions by "simpler" ones.

The simplest functions are usually **polynomials:**

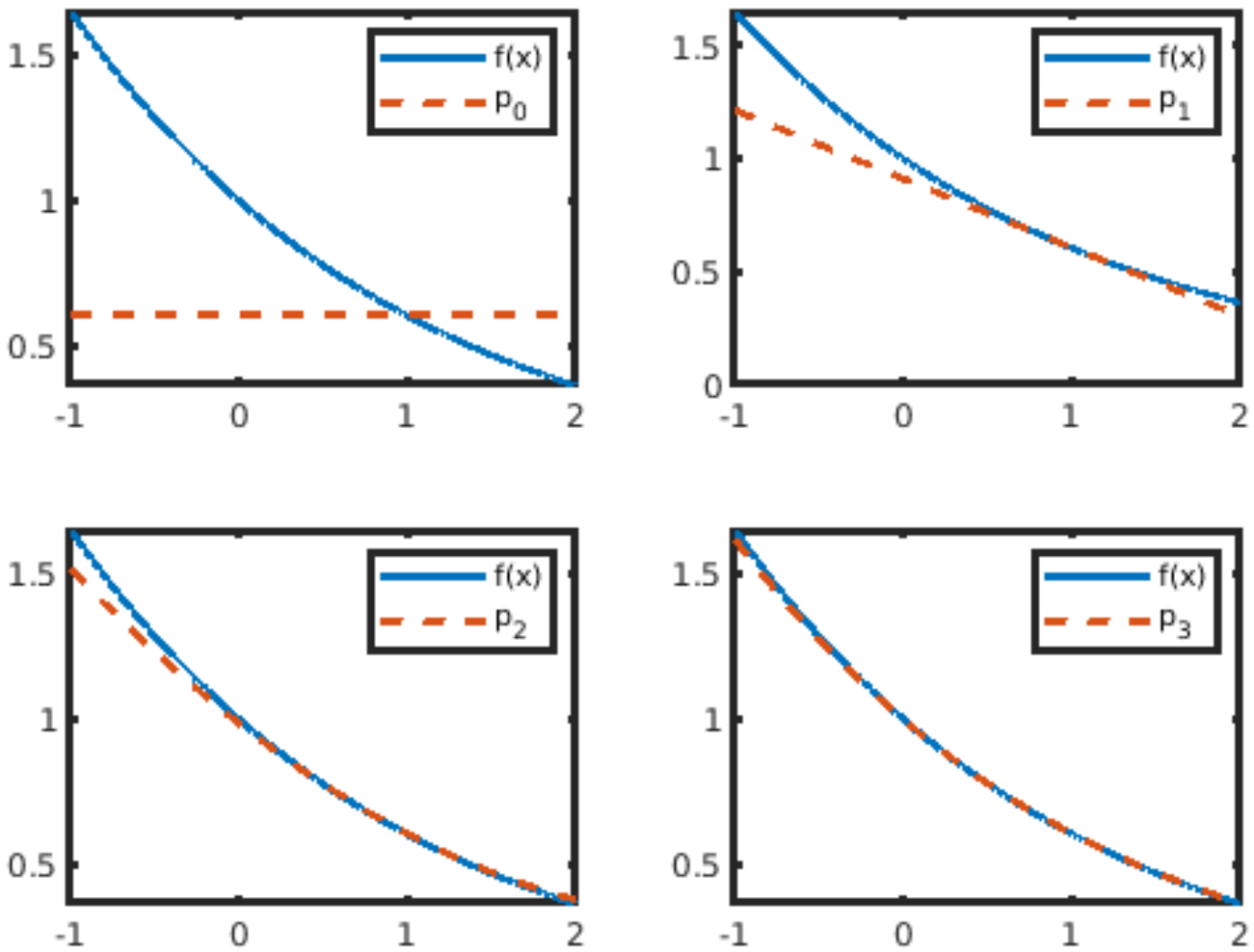
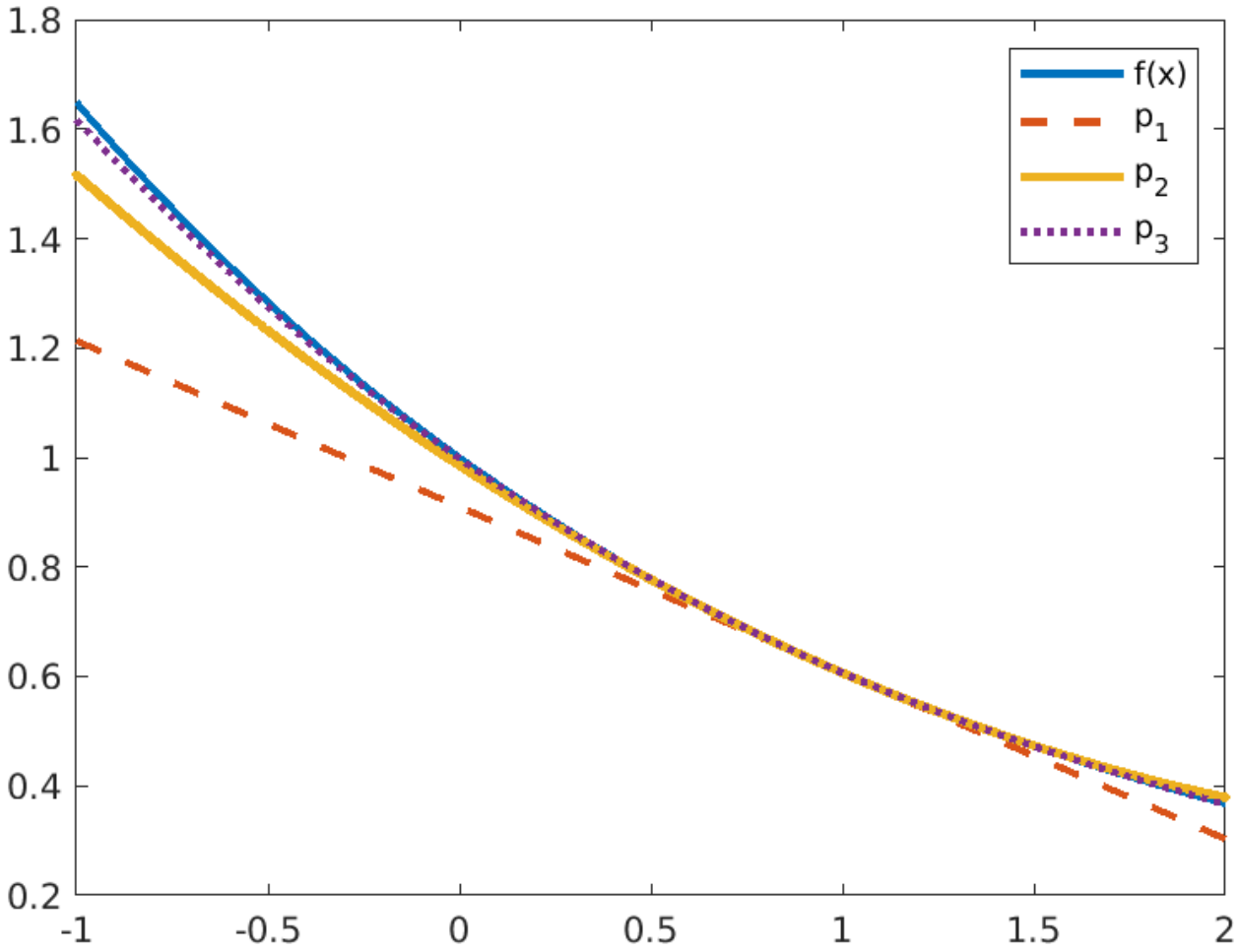
- Constants (degree zero)
- Linear (degree one), e.g.,
- Quadratic, etc, e.g.,

A **Taylor** polynomial is the most standard approximation: given a function $f(x)$, its Taylor Polynomial of order n at $x = a$ is

$$p_n(x) := f(a) + f'(a)(x-a) + \frac{1}{2}f''(a)(x-a)^2 + \frac{1}{3!}f'''(a)(x-a)^3 + \cdots + \frac{1}{n!}f^{(n)}(a)(x-a)^n.$$

TaylorPoly.m

```
f = @(x)exp(-x/2);
df = @(x)-(1/2)*exp(-x/2);
d2f = @(x)(1/4)*exp(-x/2);
d3f = @(x)-(1/8)*exp(-x/2);

a = 1.0;
p0 = @(x)(f(a) + x*0);
p1 = @(x)(p0(x) + (x-a).*df(a));
p2 = @(x)(p1(x) + (x-a).^2.*d2f(a)/2);
p3 = @(x)(p2(x) + (x-a).^3.*d3f(a)/6);

X = linspace(-1,2,1001); % Lots of points for plotting;

figure(1); subplot(2,2,1);
plot(X, f(X), X, p0(X), '--'); legend('f(x)', 'p_0')

subplot(2,2,2);
plot(X, f(X), X, p1(X), '--'); legend('f(x)', 'p_1');

subplot(2,2,3);
plot(X, f(X), X, p2(X), '--'); legend('f(x)', 'p_2')
```


f(x)
p_0
f(x)
p_1
f(x)
p_2
f(x)
p_3
1.5
1
0.5
0
-1
0
1
2

```
plot(X, f(X), X, p1(X), '--', X, p2(X), '.-', X, ...
  p3(X), ':', 'LineWidth', 2);
legend('f(x)', 'p_1', 'p_2', 'p_3')
```